

How the AI *concepts connect.*

Make sense of the names. See how they fit.

Follow one assistant from model to application.

🕒 18-22 min core reading · Beginner friendly · 12 short sections

One application can use machine learning, generate language, retrieve documents, and act through tools. These names describe different parts of the same system.

WHAT YOU WILL BE ABLE TO EXPLAIN

Place AI, ML, deep learning, NLP, GenAI, and LLMs on a clear map. Distinguish a model from RAG and an agent. Describe a practical assistant without mixing up its task, method, and architecture.

Reading path: Read the main explanations, explore the four versions of the assistant, then try the three core exercises. Expand the optional notes when useful. Chapter 1 is the only prerequisite; no coding or new mathematics is needed.

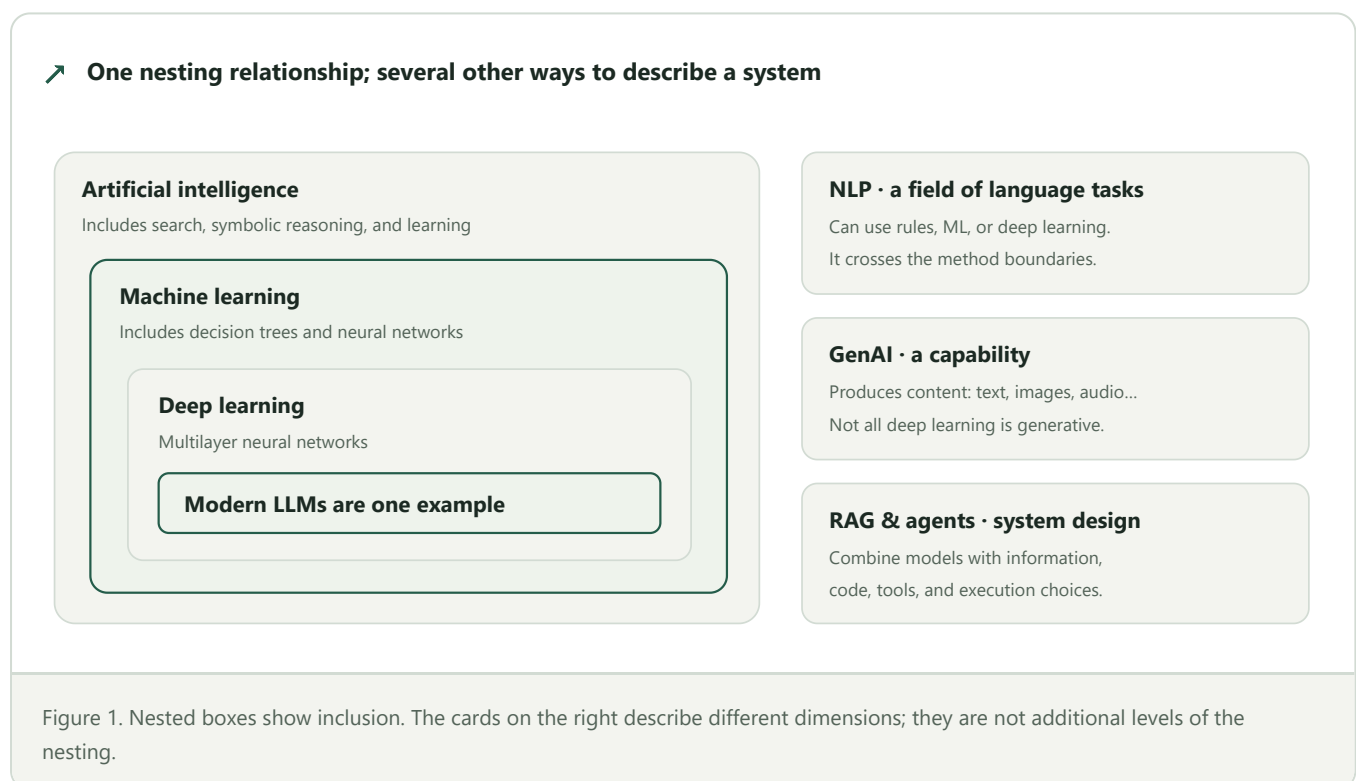
1. Start with the question each term answers

Imagine someone saying, “Our chatbot is AI, ML, deep learning, GenAI, an LLM, RAG, and an agent.” Some of these labels might fit, but they do not all name the same thing.

Ask what each label tells you:

Question	Relevant terms	What they describe
What broad field are we working in?	AI	Building systems that perform tasks associated with intelligence
How is behavior learned?	ML, deep learning	Families of learning methods
What kind of problem is being solved?	NLP	Problems involving human language
What capability is being used?	Generative AI	Producing content, such as text, images, or audio
What model is inside?	LLM	A large language model
How are components connected?	RAG, workflows, agents	Ways to supply information and organize execution
How broad is the capability we want?	AGI	A research concept concerning general intelligence

The most useful nesting relationship is **deep learning within machine learning within AI**. The other terms need different relationships.



Read the boxes as “is a kind of,” not “must come next.” A decision tree can be ML without deep learning. Language processing can use hand-written rules. An LLM application can use retrieval, tools, both, or neither.

2. Machine learning and deep learning

Machine learning (ML) fits a model using data and a learning objective. Instead of writing every decision rule yourself, you choose a learning procedure that adjusts the model from examples. You still decide what to measure and whether its performance is useful.

For a study-question classifier, the task is to predict a question's topic. A training example pairs "How do derivatives work?" with the label "calculus." A **label** is the target answer supplied for that example; a **feature** is an input value available to the model.

A model might learn from features such as question length and counts of selected words. A **decision tree** learns a sequence of branching tests on these features. It is an ML method; it does not need a neural network.

Deep learning (DL) uses neural networks with multiple layers of learned transformations. A layer converts its input values into a new representation. Training adjusts the numerical settings, called **parameters**, across the network so that its outputs better meet the objective.

For language, learned representations can encode useful patterns in words and their context. That can reduce the need to manually design every feature. It does not remove the need for suitable data, evaluation, or application logic.

Same task, different method: a decision tree and a deep neural network can both classify study questions. Neither has to generate an explanation of the answer. Choose between them using measured results, data needs, cost, and complexity. "Deep" describes the model structure; it does not guarantee deeper understanding.

▼ Go deeper: what is a learned representation?

A representation is a numerical form of information that a model can work with. A manually chosen feature might be "does the question contain the word derivative?" A learned representation can capture combinations of signals that are useful for the training objective, including context that an isolated word misses.

For example, "How do derivatives work?" and "Explain rates of change" may acquire related representations when training rewards handling them similarly. This is not guaranteed by the architecture alone. Data, objectives, and training quality matter. Later chapters explain embeddings, layers, and parameter updates mathematically.

"More layers" and "more parameters" are related design choices, but not synonyms. A layer can be wide or narrow, and different network designs use their parameters differently.

3. NLP is a field of problems

Natural language processing (NLP) concerns computational work with human language: classifying messages, finding names, translating, searching text, summarizing, and more.

NLP tells you **what the system works on**, not exactly how it works. Finding a lesson ID in a course note might use a written matching pattern. Classifying its topic might use a small ML model. Drafting a readable response might use an LLM. All involve language; only some require generation.

Consider “Lesson LES-104 takes an estimated 20 minutes.” Extracting “LES-104” and “20” into fields is an NLP task. A pattern matcher can handle this standardized input without learning from examples. More varied language may require a different method.

An LLM is one powerful tool for NLP, not a replacement name for the whole field. [Hugging Face’s NLP and LLM introduction](#) develops this distinction.

4. Generative AI and LLMs

Generative AI (GenAI) produces content from learned patterns, usually in response to some input. Examples include writing a summary, generating an image from a description, or producing speech from text.

A large language model (LLM) is a language model trained at substantial scale. “Large” has no universal numerical cutoff. Modern LLMs use deep learning; many familiar text generators use transformer networks, which we will unpack later.

An LLM represents text as **tokens**: units that may be words, word pieces, or punctuation. In the common next-token approach to generation, it estimates possibilities for the next token from the available context; a decoding procedure selects a token and continues. This mechanism can produce an explanation or code, but does not itself guarantee factual correctness.

For our assistant, “Explain overfitting using my notes” is a **prompt**, the input containing instructions and any supplied material. The **context window** is the model’s bounded working input/output capacity, measured in tokens according to the model’s limits. It is not an unlimited memory of everything the user has ever said.

Two distinctions keep this clear:

GenAI extends beyond LLMs. Image and audio generators can use other model families.

An LLM can perform a classification task. Asking it to return a topic such as “calculus” uses a generative model to produce a label. The application task is still classification.

The model is also different from the chat application around it. A chat interface may add document uploads, stored history, search, permission checks, and tool execution. Those features are not automatically inside the model.

▼ Go deeper: context, parameters, and foundation models

Parameters are numerical values fitted during training. **Context** is information supplied for a particular inference call. Pasting a new study note into context can change the answer without changing the parameters.

Fine-tuning means additional training that updates model parameters. This is different from prompting or retrieving documents. An application might store conversation history and include it in later prompts; that is not, by itself, fine-tuning.

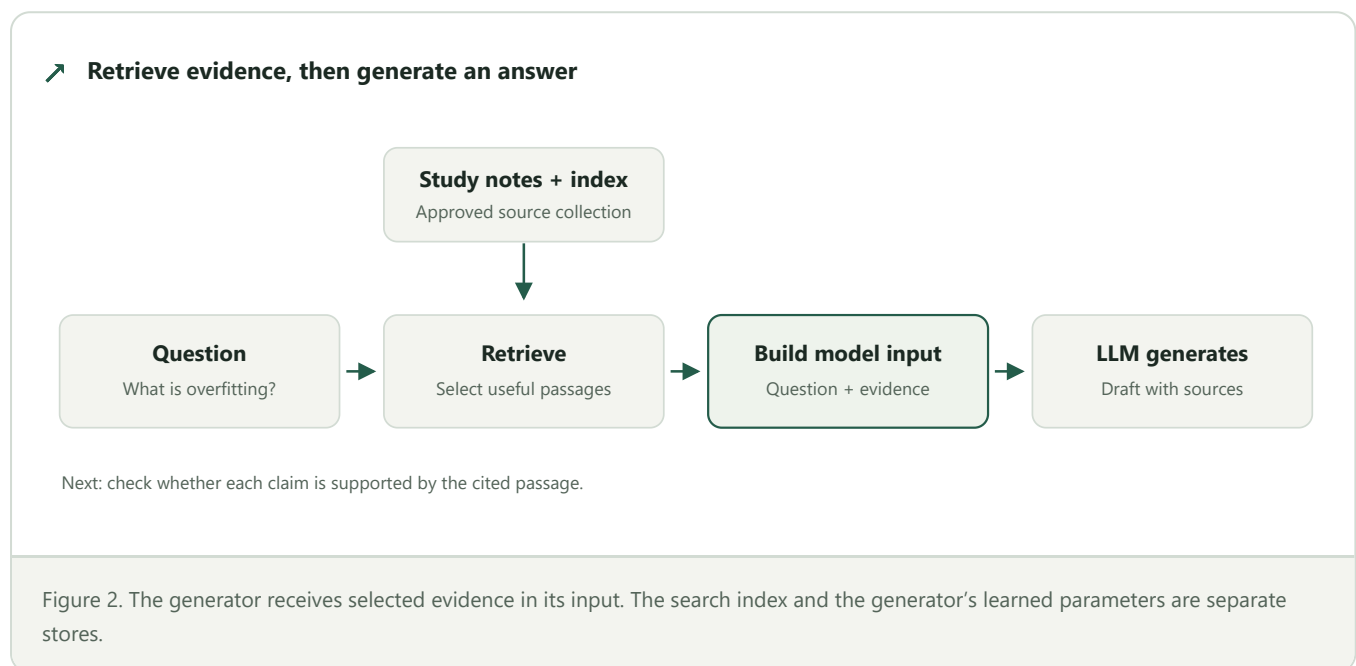
A **foundation model** is trained broadly and can be adapted to different tasks. The term is wider than LLM: it can include vision and multimodal models. A **multimodal** model handles more than one kind of information, such as text and images. Exactly which inputs and outputs it supports depends on the model.

The boundaries of “large” and “foundation” are conventions rather than a universal size test. Focus on the model’s supported tasks, measured behavior, and operating requirements.

5. RAG supplies information before generation

Suppose the user asks: “**What do my notes say about overfitting?**” A model’s training does not ensure it knows your notes or their latest revisions.

Retrieval-augmented generation (RAG) connects retrieval with generation. The application finds relevant information, includes it in the generator’s context, and asks for an answer based on that evidence.



Using Chapter 1’s fictional note N01, retrieval finds: “A model can fit training examples too closely and perform poorly on new cases.” A suitable answer explains overfitting and cites N01. It should not invent an exam date that the note never names.

Retrieval can use keywords, numerical representations called embeddings, or a combination. A vector database is one implementation option, not the definition of RAG. Here, **retrieval supplies context; it does not itself retrain the generator**. [Microsoft’s RAG overview](#) explains the retrieval-and-generation pattern.

RAG adds evidence, not certainty. Search might miss the relevant passage, retrieve an obsolete version, or return a passage the model misinterprets. Check retrieval quality and whether the answer’s claims are actually supported. A citation alone is not proof that a claim follows from its source.

▼ Go deeper: how the documents become searchable

A common preparation path is to extract document text, divide it into useful passages called **chunks**, and build an **index**, a structure used to find candidates efficiently. Keep document names, versions, and locations with each passage.

At question time, retrieval selects passages; the application constructs the model input; generation produces the answer. Preparing the collection and answering a question are separate activities. A saved index can become stale if changes in the source documents are not reflected in it.

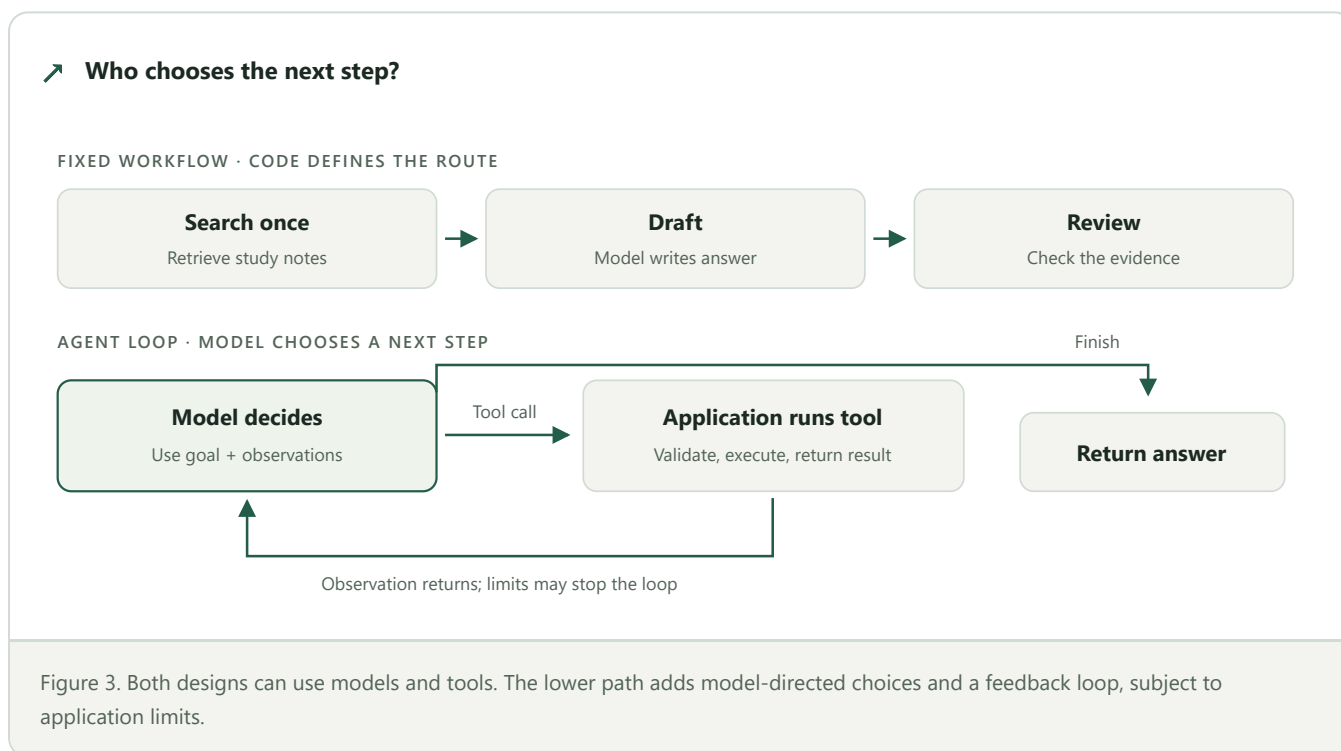
Manually pasting one passage gives a model context. It does not demonstrate an application that automatically retrieves relevant information. Likewise, search that only returns links does not contain the generation step of RAG.

6. Workflows, tools, and agents

A **tool** is a function an application makes available, such as searching study notes, reading a lesson, or adding estimated study times. When a model requests a tool call, application code validates and executes the allowed function and returns its result. The request itself is not execution.

A **workflow** organizes steps. In a fixed workflow, code defines the route: retrieve study notes, draft an answer, check the required fields, then send the draft for review. The workflow can contain model calls, conditions, and retries.

In the **LLM agent** sense used here, the model can choose the next step based on the goal and observations. It might search, inspect a lesson and its prerequisites, search again with a different phrase, then decide it has enough evidence to respond. This creates a loop.



The difference is control over the next step. One tool call, a chat interface, or a long sequence of steps does not by itself establish an agent. Definitions vary; Anthropic's engineering guide uses this distinction between predefined workflows and model-directed execution.

An agent can use RAG as part of its work. RAG can also run in a fixed workflow with no agent loop. Neither requires the other.

For our assistant, read-only access to notes may be enough. Proposing a revision schedule is different from writing calendar events: calendar changes need explicit permission, validated details, and any required review. Set stopping conditions and tool-call limits so an unsuccessful search cannot continue indefinitely.

▼ Go deeper: why “agent” has more than one meaning

In broader AI, an agent is a system that perceives an environment and acts in it. It need not contain an LLM; a game-playing system or robot controller can fit this description.

This handbook uses the narrower phrase **LLM agent** when discussing a model that helps determine the sequence of tool use and other steps. Always describe the actual control loop, permissions, and stopping rules instead of relying on the product label.

7. Put the pieces into one application

Our fictional **personal learning assistant** receives a study question, searches supplied notes, drafts an explanation with sources, and gives the learner a reviewable result. Later application chapters will extend this project with quizzes and revision planning.

➤ A model is one component of the assistant

APPLICATION · UI, PERMISSIONS, LOGGING, AND LIMITS

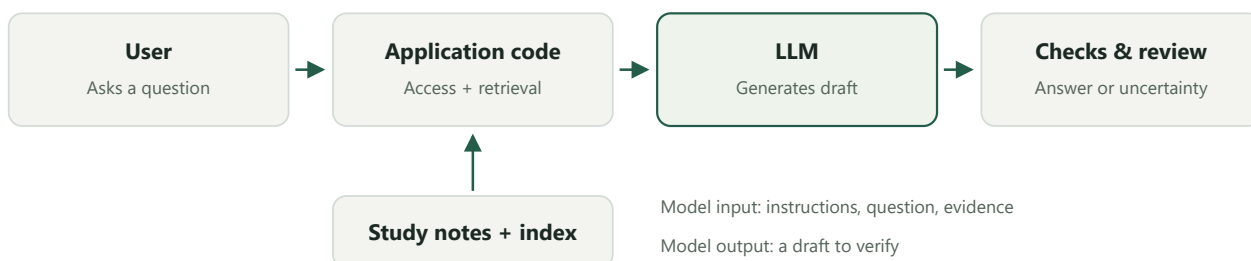


Figure 4. This is a fixed RAG workflow. Adding retrieval does not by itself add an agent loop. Access checks and review remain application responsibilities.

At different levels, it is an **AI application**, performs **NLP**, uses a **deep-learning LLM**, provides **generative** answering, and has a **RAG** architecture. Those labels can all be true simultaneously. Add “agent” only when its execution design supports that description.

The application also contains ordinary software: a user interface, identity checks, document access rules, data formatting, and logging. Naming its model does not describe all of these responsibilities.

Explore four possible designs below. These are alternatives for learning, not a ranking in which the last is always best. The examples are fixed illustrations; no model or external service is called.

Compare these four illustrative designs:

Design	What it contains	Check its result
Classify	Question → trained decision tree → topic AI, ML, NLP	Does it assign the right category on new study questions?
Draft	Question + supplied note → LLM → explanation AI, ML, Deep learning, NLP, LLM, GenAI	Does the draft preserve facts and avoid unsupported claims?
Retrieve	Question → note search → evidence + LLM → answer AI, ML, Deep learning, NLP, LLM, GenAI, RAG	Were the right passages retrieved, and do they support the answer?
Investigate	Goal → model chooses tool → observation → decide again AI, ML, Deep learning, NLP, LLM, GenAI, RAG, LLM agent	Does the loop reach a supported answer, use tools correctly, and stop?

For each design, notice **what changed**: the output task, the information supplied, or control of the next step. More components introduce more places for errors as well as more capabilities.

8. Where AGI fits

Artificial general intelligence (AGI) is a research concept concerned with broad capability across tasks and situations, rather than competence on only a restricted set. Definitions differ on the breadth, performance level, and autonomy required.

AGI is therefore not another component to add after RAG or agents. A proposed framework, [Levels of AGI](#), separates questions such as breadth and performance when describing progress. It is a framework, not a universally accepted test.

An assistant that reads lesson diagrams and writes useful explanations demonstrates those capabilities. That evidence alone does not establish general intelligence. **Multimodal** describes information types; **agentic** describes aspects of execution; neither establishes AGI.

For practical work, use narrower, testable claims: "On this held-out set of study questions, the system retrieved relevant notes and produced answers supported by those notes in these cases." A **held-out set** contains examples excluded from fitting the system, used to assess its behavior on new cases. We will examine evaluation in later chapters.

9. Choose from the need

Start by writing the required input, output, and success criterion. Then choose a baseline simple enough to understand and test.

Need in the learning assistant	A sensible starting point	What you would check
Check whether 20- and 15-minute lessons fit a 30-minute session	Ordinary arithmetic	35 minutes exceeds the budget by 5 minutes
Predict a study question's topic	Rules or a small classifier as a baseline	Errors by topic on new questions
Explain a concept from supplied notes	A template or an LLM, compared on the task	Coverage of facts and unsupported claims
Answer from changing study notes	Search; add RAG when a composed answer is needed	Retrieval relevance and evidence support
Choose follow-up searches based on results	Start with an explicit workflow; consider an agent if flexibility helps	Task success, tool mistakes, time, and cost

No technique is automatically required because another is present. A classifier does not need to write prose. A note search tool need not generate an answer. A RAG assistant need not choose its own workflow.

10. Practice: explain the design

Exercise 1: give each label a reason

A deep neural network was trained on labelled study questions to output algebra, calculus, or probability. It uses no document retrieval and does not produce free-form explanations. Which labels fit: AI, ML, DL, NLP, RAG? Does the description establish generative AI?

▼ Reveal the explanation

AI, ML, DL, and NLP fit: the system performs a language task using a learned deep neural network. RAG does not fit because no retrieval-and-generation path is described. The classification task does not establish a generative model or generative capability; do not infer one merely from the use of deep learning.

Exercise 2: trace one question

Draw how “What do my notes say about overfitting?” reaches a RAG answer. Include the document collection, retrieved passage, model input, generated answer, and evidence check. Which step changes the LLM’s parameters?

▼ Reveal the explanation

The question drives retrieval over the document collection. Selected passages join the question and instructions in the model input. The LLM generates an answer, whose claims and citations are checked against the passages. None of these ordinary inference steps requires changing the LLM’s parameters. Updating the searchable collection is also distinct from training the generator.

Exercise 3: find the control boundary

Design A always searches once, summarizes, and stops. Design B lets a model inspect results and choose between another search, reading a study note, or finishing. Which is a fixed workflow and which is an LLM agent loop? Give one useful stopping condition for B.

▼ Reveal the explanation

A is a fixed workflow; B is an LLM agent loop under our stated definition. B could stop after a set tool-call limit and return what was found with any missing evidence identified. It should also stop when sufficient evidence supports an answer. The application enforces limits; the model's decision to continue does not override them.

▼ Extra practice: test the boundaries

Exercise 4. An application retrieves study notes but displays only their titles and excerpts. Is that enough to call it RAG?

▼ Reveal the explanation

No. This describes retrieval without a generative answer. It can still be a useful search application.

Exercise 5. The same LLM first writes a paragraph, then returns a single category in another application. Did its model family change?

▼ Reveal the explanation

No. The application task and output constraints changed. A model's type and the task it is used for are different descriptions.

11. Check your understanding

Choose an answer for each question before checking. Use the explanations to revisit the distinction you missed.

01 Which relationship is accurate?

- A. Every ML model is a deep neural network.
- B. Deep learning is a family of methods within machine learning.
- C. RAG is the next model type after LLMs.

Answer: B. Machine learning also includes methods such as decision trees. RAG describes how retrieval and generation are connected in a system.

02 A program finds dates in messages using written patterns. What does that tell us?

- A. It performs an NLP task without requiring an LLM.
- B. It must have learned from labelled examples.
- C. It is a generative language model.

Answer: A. NLP describes the language problem. Written matching rules can solve some NLP tasks without model training or generation.

03 A search tool retrieves a study note and supplies it to an LLM. What changed?

- A. The LLM necessarily learned new parameters.
- B. The model became an agent automatically.
- C. The information in the model input changed.

Answer: C. This is retrieval-augmented generation. Retrieval supplies context at use time; it does not, by itself, update the generator's parameters.

04 Which design is an agent loop in the sense used here?

- A. Code always runs search, then summary, then review.
- B. A model chooses a tool, reads its result, and decides whether to search again or finish.
- C. A page shows an LLM answer in a chat bubble.

Answer: B. The distinction is who determines the next step. A model-directed loop can adapt to observations. A fixed workflow can use the same model and tools.

05 An assistant reads lesson diagrams and writes useful explanations. Does that establish AGI?

- A. No; those abilities alone do not establish broad general capability.
- B. Yes; multiple input types are sufficient.
- C. Yes; all LLM applications are AGI.

Answer: A. Multimodal describes the kinds of information processed. AGI concerns breadth and level of capability across tasks, with definitions and evaluation criteria that must be stated.

12. Keep this map; build on it next

Finish by describing an application in one sentence with separate roles:

“A learning assistant uses an LLM to explain concepts, retrieves study notes through RAG, and follows a fixed workflow with a review step.”

Replace “fixed workflow” with “agent loop” only if the model helps choose subsequent steps. You should now be able to explain why a single system can have several accurate labels, and why adding one does not automatically add the others.

▼ Quick-reference glossary

Term	Recall cue
AI	Broad field of intelligent systems
ML	Models fitted from data against an objective
Deep learning	Learning with multilayer neural networks
NLP	Computational problems involving human language
GenAI	Generating content from learned patterns
LLM	A large language model
Token	A model’s unit of text representation
Prompt / context	Supplied input / information available to the model for a call
Parameters	Numerical model settings fitted during training
Fine-tuning	Further training that updates parameters
Foundation model	A broadly trained model adaptable to multiple tasks
Multimodal	Working with multiple information types
RAG	Retrieval connected to generation
Chunk / index	A passage / a structure supporting search
Tool	An application function available for use
Workflow	An organized execution path
LLM agent	Model-directed choice of steps in a goal-oriented loop
AGI	General capability as a research concept; criteria must be stated

The scenarios, diagrams, and exercises are original learning examples. Further reading is optional:

Google: What is machine learning?

Hugging Face: NLP and LLMs

Microsoft: retrieval-augmented generation

Anthropic: workflows and agents

Levels of AGI: a proposed framework

Next: Chapter 3 explains how models learn, what an objective does, and why errors appear on new data.